ConectaStore

Sistema de recomendação de produtos baseado em grafos

Relatório técnico de Data Structure Strategy and Implementation

1 Problema e proposta

O catálogo amplo da MegaStore dificulta a descoberta de produtos. Recomendar apenas itens mais vendidos ou da mesma categoria ignora conexões entre clientes e compras e tende a produzir sugestões genéricas. O ConectaStore organiza essas relações em um grafo para produzir recomendações explicáveis a partir de um cliente ou de um produto.

A solução implementada em Rust utiliza listas de adjacência, busca em largura limitada e um ranking por evidências em caminhos mínimos. O protótipo permite cadastrar e consultar produtos, criar e remover conexões e executar recomendações pela linha de comando. A base fica em memória durante a sessão interativa.

O resultado principal é uma implementação funcional, compilada e validada por nove testes. No experimento local com 50.000 produtos, a mediana de uma consulta foi 0,4169 ms. Esse resultado caracteriza a carga sintética medida; não demonstra atendimento a milhões de produtos nem aumento de vendas.

Escopo e critérios de sucesso

Os critérios técnicos são: representar entidades e relações de forma consistente, recomendar sem duplicatas, excluir produtos já comprados, justificar o percurso e as estruturas, testar o comportamento e medir tempos com volumes distintos. O projeto não depende de crates externos e acompanha README com comandos reproduzíveis.

Não foram implementados autenticação, persistência, interface web ou treinamento de modelos. Nomes são metadados dos produtos; clientes e categorias são identificados por números. A relevância comercial precisaria de avaliação com históricos reais, usando separação temporal entre treino e teste.

Organização

As seções seguintes apresentam a modelagem, as estruturas, o algoritmo, a análise de complexidade, a validação e o desempenho. A fundamentação corresponde diretamente a Store e recommend, em src/lib.rs, e aos comandos e medições em src/main.rs.

2 Modelagem do sistema como grafo

O grafo é heterogêneo, não direcionado, ponderado e sem laços. Vértices são variantes do enum Vertex: Customer(id), Product(id) e Category(id). Assim, cliente 1 e produto 1 são entidades distintas. Relações de tipos diferentes podem coexistir entre o mesmo par, mas cada par possui no máximo uma aresta de cada tipo: trata-se de um multigrafo tipado restrito.

Relação	Extremidades permitidas	Significado
Purchase	Cliente e produto	Compra realizada
Interest	Cliente e produto ou categoria	Afinidade declarada
BelongsTo	Produto e categoria	Associação ao catálogo
Similarity	Produto e produto	Similaridade cadastrada

Cada aresta lógica é registrada nas duas listas de adjacência. Isso permite percorrer compras nos dois sentidos para encontrar consumidores que compraram produtos em comum. O sentido de negócio é recuperado pelo tipo da relação e dos vértices; não se presume que um produto realiza compras.

Os pesos são f64 finitos no intervalo (0, 1]. Na demonstração, compra e associação à categoria usam peso 1; interesse usa 0,8; similaridades têm pesos 0,9 e 0,7. São parâmetros de evidência definidos para o exemplo, sem calibração estatística. Pesos não representam distância e não são probabilidades de compra.

Caminhos que justificam uma recomendação

Cliente 1 -> Notebook -> Mouse: utiliza compra e similaridade. Cliente 1 -> Eletrônicos -> Mouse: utiliza interesse e categoria. Cliente 1 -> Notebook -> Cliente 2 -> Teclado: utiliza compras em comum e oferece uma evidência colaborativa. Esses percursos integram relações de catálogo e comportamento.

Integridade e gerenciamento

connect valida extremos, tipo e peso antes de alterar o grafo. A repetição do mesmo par e tipo atualiza o peso em ambos os sentidos. disconnect remove os dois registros. Laços, pesos inválidos e referências a vértices inexistentes retornam erro. A API não expõe mutação direta das listas, preservando a simetria.

O grafo pode conter ciclos e componentes desconectados. A BFS controla a distância de cada vértice e não volta a expandi-lo. Um cliente sem conexões retorna lista vazia; uma origem inexistente retorna erro. Avaliações não são um tipo próprio nesta versão; compra, interesse e similaridade já fornecem as relações exigidas para o protótipo.

3 Estruturas de dados e representação

Aspecto	Lista de adjacência	Matriz de adjacência
Espaço estrutural	O(V + E)	O(V²) para tipos fixos
Percorrer vizinhos de u	O(grau(u))	O(V)
Consultar aresta u e v	O(grau(u)) com vetor	O(1)
Inserir sem duplicação	Varre listas dos extremos	O(1) com índices existentes
Catálogo esparso	Armazena conexões existentes	Reserva posições sem conexão

Uma matriz simplifica consultas por par, mas é inadequada para um catálogo grande e esparso. Com um milhão de vértices, seriam 10^{12} células: apenas um byte por célula já corresponderia a cerca de 1 TB decimal, sem pesos nem tipos. Não é uma estimativa de memória do programa; é uma comparação de ordem de grandeza.

A implementação usa HashMap<Vertex, Vec<Edge>>. Cada Vec agrupa os vizinhos de um vértice e cada Edge guarda destino, tipo e peso. Há duas entradas para cada aresta lógica. O vetor favorece percursos sequenciais, mas atualizar ou remover uma conexão exige procurar nela. Um mapa adicional de arestas poderia reduzir a busca por par, consumindo mais memória.

Estrutura concreta	Função na implementação
HashMap<u32, Product>	Consulta de produto por ID com custo esperado O(1)
HashMap<Vertex, Vec<Edge>>	Listas de adjacência e existência dos vértices
VecDeque<Vertex>	Fila FIFO da busca em largura
HashMap<Vertex, usize>	Distância e descoberta única de cada vértice
HashMap<Vertex, f64>	Evidência acumulada pelos caminhos mínimos
HashSet<u32>	Exclusão de produtos comprados e da origem
Vec<Recommendation>	Ordenação e truncamento dos candidatos

A documentação oficial de collections [1] fundamenta os custos esperados dos mapas e a escolha de VecDeque para filas. As análises deste relatório consideram hashing esperado, e não acesso constante garantido no pior caso. A memória real também inclui capacidade ociosa, buckets, alocações e os bytes dos nomes dos produtos.

4 Algoritmo de recomendação

A BFS foi escolhida porque a proximidade do grafo é definida pelo número de relações atravessadas. A fila visita os níveis em ordem crescente e permite limitar a profundidade sem explorar caminhos completos individualmente. DFS não fornece essa ordem naturalmente; Dijkstra seria adequado se os pesos fossem custos de percurso, o que não ocorre nesta modelagem.

Etapas implementadas

1. Validar a origem. Se for cliente, reunir os produtos ligados por Purchase no conjunto de excluídos; se for produto, excluir seu próprio ID. Produtos comprados continuam participando dos percursos como evidências, embora não sejam candidatos.

2. Inicializar distância da origem em zero, evidência em 1 e uma fila com a origem. Retirar um vértice por vez e não expandi-lo quando atingir a profundidade máxima. Vértices na fronteira ainda podem ser candidatos.

3. Para cada aresta examinada, descobrir o destino e colocá-lo na fila somente na primeira visita. Se sua distância for exatamente a distância atual mais um, acumular evidência multiplicada pelo peso da aresta. Arestas para o mesmo nível ou níveis anteriores são ignoradas na pontuação.

4. Transformar os produtos alcançados e não excluídos em candidatos. Calcular a pontuação, ordenar em ordem decrescente e desempatar por ID crescente. Aplicar o limite de resultados somente depois dessa ordenação.

Definição da pontuação

Seja d(v) a menor quantidade de saltos a partir da origem. Defina e(origem)=1. Para cada predecessor u com d(u)+1=d(v), acumule e(v) += e(u) × w(u,v). Para um produto candidato, score(v)=e(v)/d(v). Isso equivale à soma dos produtos de pesos dos caminhos mínimos, dividida pela distância.

A ordem FIFO garante que todos os vértices do nível anterior contribuam antes da expansão de um destino. A recorrência evita enumerar um número potencialmente grande de caminhos. A soma pode ultrapassar 1; score mede evidência agregada. Não representa probabilidade nem um caminho de peso máximo.

Exemplo verificável e limites

Para o cliente 1, Mouse recebe 1 × 0,9 pelo notebook e 0,8 × 1 pela categoria. Sua distância é 2 e o score é (0,9+0,8)/2=0,85. Monitor recebe (0,7+0,8)/2=0,75; Teclado recebe 0,8/2=0,40. O caminho colaborativo até Teclado tem três saltos e não é somado, pois existe um caminho de dois saltos.

O mapa de distâncias gera cada candidato uma única vez. Caminhos mais longos e ciclos não reforçam o ranking. A soma favorece itens com muitas conexões, e pesos de relações distintas não são calibrados. Um cliente novo sem interesses ou compras não recebe recomendações; um fallback de popularidade seria uma evolução.

5 Complexidade e escalabilidade

Considere V vértices, E arestas lógicas, d0 o grau da origem, Vd os vértices descobertos na consulta, Ed as entradas de adjacência examinadas antes da fronteira e C os produtos elegíveis. As duas cópias por aresta não alteram a ordem assintótica.

Operação	Tempo esperado
Cadastrar ou consultar produto	O(1) para índices, mais cópia do nome no cadastro
Conectar ou desconectar u e v	O(grau(u) + grau(v))
Listar produtos em ordem de ID	O(P log P), com P produtos
Preparar exclusões e executar BFS	O(d0 + Vd + Ed)
Ordenar candidatos e aplicar limite	O(C log C)
Consulta completa	O(d0 + Vd + Ed + C log C)

O espaço auxiliar da consulta é O(d0+Vd+C), para exclusões, mapas, fila e resultado. A estrutura persistente em memória usa O(V+E+L), em que L é o total de bytes dos nomes. No pior caso, uma consulta percorre todo o grafo e sua ordenação pode custar O(V log V). Hashing e realocações explicam a qualificação de custos esperados ou amortizados.

Um limite de dez resultados não limita a exploração: todos os candidatos alcançáveis na profundidade definida são pontuados e ordenados. Limitar saltos também não garante pouco trabalho quando uma categoria é um hub com muitos vizinhos. Isso explica por que consultas ficam mais lentas com o crescimento da base sintética.

Custo de construção e crescimento

O construtor chama connect repetidamente. Como cada chamada verifica duplicatas varrendo vetores, a construção não tem garantia O(V+E). Uma categoria que recebe m produtos pode exigir $O(m^2)$ comparações ao longo de suas inserções. A medição inclui esse custo. Para bases maiores, convém usar índices de arestas na ingestão ou deduplicar em lote antes de construir vetores.

Uma evolução para milhões de produtos combina representação compacta como CSR para partes estáticas, mapas para alterações recentes, seleção de vizinhos de hubs, cache por origem com invalidação e atualização incremental. Controle de fan-out muda os candidatos explorados e deve ter seu efeito na qualidade medido. Particionamento precisa considerar arestas entre partições e latência de comunicação.

Um heap de tamanho k pode substituir a ordenação integral e reduzir o ranking para O(C log k), mantendo o custo da BFS. Normalização por grau e decaimento temporal podem reduzir viés de popularidade. Essas melhorias são propostas de evolução e não fazem parte dos resultados medidos do protótipo.

6 Validação e medição de desempenho

A execução de cargo test concluiu seis testes unitários e três testes de integração, sem falhas. Os unitários verificam cadastro, consultas, validação de arestas, atualização sem duplicação, remoção simétrica, isolamento, limites zero, ciclos, pesos, acúmulo de caminhos mínimos e exclusão dos comprados. Um cenário com duas categorias valida score 0,8 e desempate pelo menor ID.

A integração exercita o fluxo cliente -> produto -> outro cliente -> produto, seguido de inserção e remoção de similaridade. Outros dois testes iniciam o executável real: a demonstração deve produzir produtos e scores, e uma consulta a cliente inexistente deve terminar com erro. A separação entre testes de módulo e integração acompanha a organização de testes descrita no livro oficial de Rust [2].

Protocolo experimental

Ambiente: Windows x64, AMD Ryzen 7 5700G, rustc 1.99.1, alvo GNU, perfil release. O gerador cria n produtos, 100 categorias, n/5 clientes, cinco compras por cliente, uma categoria por produto e similaridade entre produtos consecutivos. Os três volumes usam a mesma regra, sem dados comerciais reais.

Para cada tamanho, mede-se a construção com Instant, executam-se cinco consultas de aquecimento e 31 consultas medidas para clientes 0 a 30, com profundidade 3 e limite 10. A consulta inclui filtragem, percurso, pontuação e ranking; não inclui criação do grafo nem impressão. black_box consome os resultados. Mediana é a 16ª amostra ordenada; p95 usa nearest rank e é a 30ª amostra.

Produtos	Vértices	Arestas	Construção ms	Mediana ms	p95 ms
1.000	1.300	2.999	0,7102	0,0129	0,0140
10.000	12.100	29.999	10,1173	0,0910	0,1541
50.000	60.100	149.999	58,6340	0,4169	0,4228

Cada volume produziu 310 recomendações no total das 31 consultas. Os registros estão em resultados/benchmark.csv. Para reproduzir, execute cargo run --release -- benchmark dentro de megastore. Hashes aleatórios, processos concorrentes e hardware podem alterar as medidas, embora a topologia gerada seja determinística.

Interpretação e limites do experimento

A mediana aumentou com o volume, coerente com categorias de grau crescente e maior quantidade de vizinhos examinados. A amostra de 31 consultas e uma rodada local caracteriza uma medição básica. Não foram medidos consumo residente de memória, vazão concorrente, latência de rede, taxa de acerto ou impacto em vendas. Não se extrapola o p95 observado para uma operação em produção.

7 Execução e conclusão técnica

Arquivos e execução

src/lib.rs concentra o modelo, as regras de integridade, as buscas e a geração de dados. src/main.rs contém comandos avulsos, sessão interativa, demonstração e benchmark. tests/workflow.rs concentra testes externos da biblioteca e do binário. Cargo.toml e Cargo.lock definem o pacote sem crates externos. O README detalha os comandos e os resultados.

Na pasta megastore, cargo build --release compila; cargo test valida; cargo run --release -- demo demonstra recomendações por cliente e por produto; cargo run --release -- interativo permite cadastrar e manter alterações durante a sessão. O comando recomendar recebe tipo da origem, ID, profundidade e limite, nessa ordem.

Um exemplo é cargo run --release -- recomendar cliente 1 3 10. O retorno contém Mouse, Monitor e Teclado, com scores 0,85, 0,75 e 0,40. A consulta por produto 1 com profundidade 2 também encontra itens conectados por categorias e compras compartilhadas. O ranking pode colocar um produto a dois saltos acima de um vizinho direto, pois soma múltiplas evidências.

Resposta ao desafio

Grafos permitem combinar relações de compra, interesse, categoria e similaridade no mesmo mecanismo de percurso. Listas de adjacência evitam reservar posições para pares inexistentes; HashMap indexa entidades e estado da consulta; VecDeque mantém a ordem BFS; HashSet filtra itens conhecidos e vetores organizam o ranking. A combinação entrega recomendações explicáveis e sem repetição no protótipo.

Para crescer, a principal decisão é controlar o subgrafo explorado e melhorar a ingestão das conexões, acompanhando qualidade e latência. A BFS limitada é adequada como base didática e funcional; as medidas locais sustentam o funcionamento no volume testado, enquanto os custos de hubs e as limitações de relevância indicam os próximos pontos de evolução.

Referências

[1] Rust Standard Library. Module std::collections. Documentação oficial. https://doc.rust-lang.org/std/collections/index.html

[2] The Rust Programming Language. Writing Automated Tests. Documentação oficial. https://doc.rust-lang.org/book/ch11-00-testing.html

[3] Enunciado da disciplina Data Structure Strategy and Implementation. ConectaStore Sistema de Recomendação de Produtos Baseado em Grafos. Material fornecido para o trabalho.